

Spécification du sous-projet autolisp-vector

OpenAI

April 27, 2026

Contents

1	Objet	3
2	Principes de conception	3
2.1	Représentation externe	3
2.2	Représentation interne	4
2.3	Allocation	4
2.4	Mutabilité	4
3	Objectifs de complexité	5
3.1	Obligatoires	5
3.2	Non-objectifs de v1	5
4	Sémantique générale	5
4.1	Longueur physique et longueur logique	5
4.2	Valeur initiale des slots	6
5	API inspirée de Common Lisp	6
5.1	API retenue en v1	6
5.1.1	Création	6
5.1.2	Accès	6
5.1.3	Mutation	7
5.1.4	Prédicats et introspection	7
5.2	API explicitement hors périmètre en v1	7
5.3	Adaptation à AutoLISP	8

6	Contrat fonctionnel détaillé	9
6.1	Création	9
6.1.1	(make-array length &key initial-element initial-contents fill-pointer)	9
6.1.2	(vector e1 e2 ... ek)	10
6.2	Accès en lecture	10
6.2.1	(aref vector index)	10
6.3	Accès en écriture	10
6.3.1	écriture d'une case	10
6.3.2	(replace vector sequence &key start1 end1 start2 end2)	11
6.4	Fill pointer	11
6.4.1	(fill-pointer vector)	11
6.4.2	mise à jour du fill pointer	12
6.5	Opérations de pile	12
6.5.1	(vector-push item vector)	12
6.5.2	(vector-pop vector)	12
6.6	Extraction	13
6.6.1	(subseq vector start &optional end)	13
6.7	Impression	13
6.7.1	(print-vector vector)	13
7	Invariants internes	14
8	Mesure et benchmarks	15
9	Baseline de performance au 14 mars 2026	15
9.1	Protocole retenu	16
9.2	Résultats synthétiques	16
9.2.1	Longueur	17
9.2.2	Lecture d'un élément	17
9.2.3	Écriture d'un élément	17
9.2.4	Opérations de pile	18
9.2.5	Sous-séquences	18
9.2.6	Remplacement de tranche	18
9.3	Lecture d'ensemble	19
9.4	Interprétation technique	19
9.5	Décision de gel	20

10 Plan de réalisation	20
10.1 Étape 1	20
10.2 Étape 2	20
10.3 Étape 3	20
10.4 Étape 4	21

1 Objet

Le sous-projet **autolisp-vector** vise à fournir une implantation de **vecteurs** pour AutoLISP avec un accès en pire cas en $O(\log(n))$, où **n** est la longueur du vecteur.

Cette première étape prépare le terrain pour une future **table de hashage** :

- les données seront indexées par chemin dans un arbre ;
- l'implantation devra permettre une étude asymptotique claire ;
- des benchmarks compareront plus tard plusieurs facteurs de branchement et plusieurs stratégies d'implantation.

La v1 se concentre sur un **vecteur fixe** non adjustable, avec support optionnel d'un **fill-pointer**.

2 Principes de conception

2.1 Représentation externe

Une instance de vecteur est représentée par un **symbole**.

Le symbole porte toutes les données de l'instance sur sa **plist**.

Tous les symboles globaux introduits par le module, publics comme internes, doivent être préfixés par **av-** ou **av--** afin d'éviter toute collision avec des symboles AutoLISP existants ou avec d'autres sous-projets du dépôt.

Exemple de schéma de propriétés :

av:kind marqueur de type, valeur attendue **vector** ;

av:length capacité totale allouée ;

av:fill-pointer taille logique, ou **nil** si absent ;

av:branching-factor facteur de branchement de l'arbre ;

av:height hauteur de l'arbre d'indexation ;

av:root racine de l'arbre contenant les cases.

Le nom du symbole doit être approprié et stable, mais la stratégie exacte de génération des noms est laissée à l'implémentation.

2.2 Représentation interne

Le stockage repose sur un arbre enraciné, adressé par décomposition de l'indice dans une base B , où B est le facteur de branchement.

Pour un indice i , l'accès consiste à :

1. décomposer i en chiffres de base B ;
2. suivre les branches correspondantes depuis la racine ;
3. atteindre une feuille qui contient le slot demandé.

La `v1` retient un **arbre binaire** ($B = 2$) comme implantation de référence :

- c'est la structure la plus simple à valider ;
- elle garantit bien $O(\log(n))$ en pire cas ;
- elle servira de baseline pour les benchmarks.

Une évolution ultérieure pourra introduire un facteur de branchement supérieur sans changer l'API publique.

2.3 Allocation

Tous les slots sont alloués à la création du vecteur, même si :

- le **fill-pointer** vaut 0 ;
- aucun élément n'a encore été écrit.

La longueur physique est donc fixée dès l'allocation.

2.4 Mutabilité

La `v1` autorise la mutation du contenu des cases ainsi que du **fill-pointer**, mais pas la modification de la capacité.

3 Objectifs de complexité

3.1 Obligatoires

lecture d'une case $O(\log(n))$ pire cas ;

écriture d'une case $O(\log(n))$ pire cas ;

lecture de la longueur $O(1)$;

lecture du fill-pointer $O(1)$;

mise à jour du fill-pointer $O(1)$;

création du vecteur $O(n)$, puisque tous les slots doivent être alloués.

3.2 Non-objectifs de v1

- minimisation absolue de la constante cachée ;
- compaction mémoire ;
- resize incrémental ;
- persistance structurelle ;
- partage entre vecteurs.

4 Sémantique générale

4.1 Longueur physique et longueur logique

Chaque vecteur a :

- une **longueur physique** fixée, notée **length** ;
- une **longueur logique** optionnelle, notée **fill-pointer**.

Si le **fill-pointer** est absent, la longueur logique est assimilée à la longueur physique pour les opérations de pile.

Si le **fill-pointer** est présent :

- il doit vérifier $0 < \text{fill-pointer} \leq \text{length}$;
- les cases au-delà du **fill-pointer** existent quand même ;

- `aref` reste défini sur toute la longueur physique ;
- les opérations de type `vector-push` et `vector-pop` utilisent la longueur logique.

4.2 Valeur initiale des slots

Chaque slot reçoit une valeur initiale explicite à la création :

- soit `initial-element` fourni par l'appelant ;
- soit une valeur par défaut documentée par l'implémentation.
- soit `:initial-contents`, qui surcharge l'initialisation case par case.

Quand `:initial-contents` est fourni, l'implantation recommandée est :

1. créer le vecteur avec allocation complète ;
2. initialiser tous les slots à une valeur par défaut ou à `initial-element` ;
3. copier `initial-contents` dans le vecteur avec `replace`.

5 API inspirée de Common Lisp

Le but est de reprendre l'**API des vecteurs Common Lisp** en sélectionnant un sous-ensemble réaliste pour AutoLISP.

5.1 API retenue en v1

5.1.1 Création

- `make-array`, limité au cas des vecteurs à une dimension ;
- `vector`, constructeur pratique à partir d'une liste d'éléments.

5.1.2 Accès

- `aref` ;
- `svref` comme alias spécialisé pour vecteur simple ;
- `length` ou `vector-length` selon ce qui s'intègre le mieux avec le reste du dépôt ;

- `fill-pointer` ;
- `copy-vector` ;
- `subseq`.

5.1.3 Mutation

- écriture d'une case, équivalent de `(setf (aref ...))` ;
- mise à jour de `fill-pointer` ;
- `replace` ;
- `vector-push` ;
- `vector-pop` ;
- `vector-push-extend` présent dans l'API mais **non supporté** en v1 car le vecteur n'est pas ajustable.

5.1.4 Prédicats et introspection

- `vectorp` ;
- `arrayp` éventuellement comme alias documentaire si le projet veut se placer dans le vocabulaire Common Lisp ;
- `array-total-size` ;
- `adjustable-array-p`, qui doit toujours retourner faux en v1 ;
- `print-vector` pour produire une représentation lisible en syntaxe Common Lisp.

5.2 API explicitement hors périmètre en v1

- tableaux multidimensionnels ;
- `adjust-array` ;
- tableaux displaced ;
- `array-displacement` ;
- `array-dimension` pour autre chose qu'un vecteur simple ;

- `array-element-type` avec sémantique Common Lisp complète ;
- `bit-vector` spécialisé ;
- `simple-vector-p` distinct de `vectorp` si cette distinction n’a pas d’intérêt pratique en AutoLISP ;
- `vector-push-extend` ;
- `map` et `map-into` ;
- support de `:initial-contents` imbriqué ou paresseux au-delà d’une séquence plate.

5.3 Adaptation à AutoLISP

AutoLISP ne fournit pas nativement toute la machinerie Common Lisp, en particulier les **generalized variables** de `setf`.

La v1 peut donc exposer une API concrète légèrement adaptée, par exemple :

- `av-make-array` ;
- `av-aref` ;
- `av-set-aref` ;
- `av-fill-pointer` ;
- `av-copy-vector` ;
- `av-replace` ;
- `av-subseq` ;
- `av-vector-push` ;
- `av-vector-pop` ;
- `av-print-vector`.

La documentation et les benchmarks devront toujours faire le lien avec les noms Common Lisp correspondants.

6 Contrat fonctionnel détaillé

6.1 Création

6.1.1 `(make-array length &key initial-element initial-contents fill-pointer)`

Contraintes :

- `length` est un entier ≥ 0 ;
- seul le cas une dimension est accepté ;
- si `fill-pointer` est fourni comme entier, il doit être compris entre 0 et `length` ;
- si `fill-pointer` vaut vrai sans entier explicite, la convention adoptée devra être documentée ; par défaut recommandé : initialiser à `length` ;
- `initial-element` et `initial-contents` ne doivent pas être fournis ensemble ;
- si `initial-contents` est fourni, sa longueur doit être égale à `length`.

Effets :

- alloue tous les slots ;
- initialise toutes les cases ;
- construit l'arbre complet requis pour adresser les indices de 0 à `length - 1` ;
- si `initial-contents` est fourni, copie son contenu dans le vecteur, par une opération sémantiquement équivalente à `(replace vector initial-contents)` ;
- retourne le symbole représentant le vecteur.

Remarque :

- `initial-contents` est traité comme une séquence plate ; la v1 n'a pas à supporter les cas multidimensionnels de Common Lisp.

6.1.2 `(vector e1 e2 ... ek)`

Effets :

- crée un vecteur de longueur `k` ;
- remplit les slots avec les éléments fournis, équivalent à `(make-array k :initial-contents '(e1 ... ek))` ;
- retourne le vecteur.

6.2 Accès en lecture

6.2.1 `(aref vector index)`

Contraintes :

- `index` doit vérifier $0 < \text{index} < \text{length=}$.

Résultat :

- retourne la valeur stockée à l'indice demandé.

Erreur :

- signalement d'erreur si l'objet n'est pas un vecteur ;
- signalement d'erreur si l'indice est hors bornes.

6.3 Accès en écriture

6.3.1 écriture d'une case

La sémantique visée est celle de :

`(setf (aref vector index) value)`

Comme `setf` n'est pas supposé disponible nativement, l'implantation peut fournir une fonction dédiée.

Contraintes :

- `index` doit vérifier $0 < \text{index} < \text{length=}$.

Effets :

- remplace la valeur du slot ;
- ne modifie ni la longueur ni le `fill-pointer`.

6.3.2 (replace vector sequence &key start1 end1 start2 end2)

But :

- copier une tranche de **sequence** dans une tranche du vecteur destination.

Périmètre v1 :

- destination obligatoire : un vecteur **autolisp-vector** ;
- source acceptée : au minimum une liste AutoLISP et un autre **autolisp-vector** ;
- les bornes optionnelles suivent l'esprit Common Lisp, avec défauts raisonnables.

Effets :

- remplace les slots de la destination dans l'intervalle demandé ;
- retourne la destination.

Complexité :

- $O(k \log(n))$ en v1, où k est le nombre d'éléments copiés.

Usage privilégié :

- initialisation de **make-array** quand **:initial-contents** est fourni ;
- copie entre vecteurs ;
- mise à jour d'une tranche.

6.4 Fill pointer

6.4.1 (fill-pointer vector)

Résultat :

- retourne le **fill-pointer** si présent ;
- sinon retourne **nil** ou une convention équivalente documentée.

6.4.2 mise à jour du fill pointer

Effets :

- fixe la taille logique ;
- ne change pas la taille physique ;
- n'alloue aucun slot supplémentaire.

6.5 Opérations de pile

6.5.1 (vector-push item vector)

Précondition :

- le vecteur possède un `fill-pointer`.

Effets :

- si `fill-pointer < length`, écrit `item` dans le slot logique courant puis incrémente le `fill-pointer` ;
- sinon échoue sans redimensionnement.

Résultat :

- retour compatible Common Lisp recommandé : l'ancien indice du nouvel élément ;
- en cas d'échec, retourner `nil` est acceptable si la convention est documentée.

6.5.2 (vector-pop vector)

Précondition :

- le vecteur possède un `fill-pointer` strictement positif.

Effets :

- décrémente le `fill-pointer` ;
- retourne l'élément précédemment au sommet logique.

Remarque :

- la valeur physique restant dans le slot libéré est laissée inchangée en `v1`.

6.6 Extraction

6.6.1 (`subseq vector start &optional end`)

But :

- extraire une sous-séquence contiguë du vecteur.

Résultat :

- retourne un nouveau `autolisp-vector` contenant les éléments des indices `start` inclus à `end` exclu ;
- si `end` est omis, il vaut la longueur physique du vecteur.

Contraintes :

- $0 < \text{start} \leq \text{end} \leq \text{length}$.

Sémantique :

- le résultat est un nouveau vecteur indépendant ;
- le `fill-pointer` du résultat est, par défaut recommandé, absent sauf si une convention plus utile est choisie et documentée.

Complexité :

- $O(k \log(k) + k \log(n))$ acceptable en v1, avec $k = \text{end} - \text{start}$.

6.7 Impression

6.7.1 (`print-vector vector`)

But :

- produire une représentation textuelle lisible en syntaxe vectorielle Common Lisp.

Format :

- la représentation des éléments suit le format `#(e0 e1 ... en-1)` ;
- l'impression doit porter sur toute la longueur physique du vecteur ;

- les espaces sont normalisés à un seul séparateur entre éléments.

Valeur de retour :

- pour rester idiomatique en AutoLISP, `print-vector` écrit la représentation et retourne ensuite l'objet vecteur lui-même.

Exemple attendu :

```
(print-vector (vector 1 2 3 4))
```

Affichage :

```
 #(1 2 3 4)
```

Valeur retournée :

```
--> %vector-3124
```

7 Invariants internes

Les invariants suivants doivent être vrais après toute opération réussie :

- le symbole possède bien le marqueur `av:kind` ;
- `av:length` est un entier ≥ 0 ;
- `av:fill-pointer` est soit absent, soit un entier dans $[[0, \text{length}]]$;
- l'arbre est assez profond pour adresser tout indice valide ;
- chaque feuille correspond à un slot effectivement alloué ;
- aucun slot n'est créé à la demande après l'initialisation ;
- l'accès à un indice donné suit un chemin déterministe unique.

8 Mesure et benchmarks

Les benchmarks n'appartiennent pas à la v1 de l'API, mais la conception doit les préparer.

Une infrastructure de benchmarks optionnelle peut être exécutée à la fin des tests via l'environnement :

`AV_RUN_BENCHMARKS=1` active les benchmarks ;

`AV_BENCH_SAMPLES=1000` fixe le nombre d'indices aléatoires utilisés pour les mesures d'accès et d'écriture.

Mesures à prévoir ensuite :

- coût de création pour différentes tailles ;
- coût moyen et pire cas de lecture ;
- coût moyen et pire cas d'écriture ;
- coût de `replace` selon la taille de tranche ;
- coût de `subseq` selon la taille extraite ;
- comparaison arbre binaire versus facteur de branchement plus grand ;
- comparaison avec une représentation `alist/plist` naïve ;
- impact du `fill-pointer` sur les opérations de pile ;
- comparaison avec les équivalents liste pour `nth`, écriture par reconstruction de liste, `length`, `cons/cdr`, extraction de sous-liste et remplacement d'une tranche au milieu.

Tailles minimales à benchmarker plus tard :

- petites tailles : 0, 1, 2, 8, 16 ;
- tailles intermédiaires : 32, 64, 128, 256 ;
- tailles plus grandes : 1024, 4096, 16384.

9 Baseline de performance au 14 mars 2026

Cette section fige l'état de la v1 après stabilisation de l'API, des tests et du banc de mesure `bench-bricscad`.

9.1 Protocole retenu

Les mesures sont produites par :

- `make -C /Users/pjb/works/sncf-reseau/src/outils-autolisp bench-bricscad`
- exécution BricsCAD avec chargement du module, des tests et des benchmarks ;
- tailles testées : 5 10 50 100 500 1000 5000 ;
- échantillons par défaut : 1000 ;
- sous-séquences et remplacements au milieu de la séquence ;
- comparaison systématique entre opérations `av-` et opérations sur listes.

Le run de référence daté du **14 mars 2026** est valide fonctionnellement :

- 14 tests unitaires OK ;
- 0 FAIL ;
- 0 ERROR ;
- benchmarks exécutés jusqu'à BENCH-END.

9.2 Résultats synthétiques

Taille	av-length	length	av-aref	nth	av-set-aref	list-set-at	av-vector-push-pop	cons-cdr	a
5	4	1	13	0	15	2	1	0	
10	4	0	13	0	16	2	1	0	
50	3	0	15	0	19	7	6	0	
100	3	0	16	0	24	10	14	0	
500	4	0	21	0	22	49	74	0	
1000	4	1	19	1	23	91	143	0	
5000	4	6	29	3	28	510	142	0	

Temps exprimés en millisecondes.

9.2.1 Longueur

`av-length` lit la longueur en temps constant via les métadonnées du vecteur. La mesure confirme le comportement attendu :

- à petite taille, l'écart avec `length` est négligeable ;
- à grande taille, `av-length` devient meilleur que `length`.

Exemples du run de référence :

- taille 1000 : `av-length` 4 ms, `length` 1 ms ;
- taille 5000 : `av-length` 4 ms, `length` 6 ms.

Conclusion : l'objectif $O(1)$ pour la longueur est atteint et visible.

9.2.2 Lecture d'un élément

`av-aref` reste plus coûteux que `nth`, mais la nouvelle représentation interne réduit très fortement la constante cachée.

Exemples du run de référence :

- taille 100 : `av-aref` 16 ms, `nth` 0 ms ;
- taille 1000 : `av-aref` 19 ms, `nth` 1 ms ;
- taille 5000 : `av-aref` 29 ms, `nth` 3 ms.

Conclusion : `av-aref` n'est pas encore compétitif face à la lecture native sur liste, mais il a quitté la zone de coût prohibitive observée sur la baseline précédente.

9.2.3 Écriture d'un élément

`av-set-aref` devient compétitif puis meilleur que la reconstruction d'une liste quand la taille augmente.

Exemples du run de référence :

- taille 100 : `av-set-aref` 24 ms, `list-set-at` 10 ms ;
- taille 500 : `av-set-aref` 22 ms, `list-set-at` 49 ms ;
- taille 5000 : `av-set-aref` 28 ms, `list-set-at` 510 ms.

Conclusion : le chemin d'écriture commence à justifier la structure arborescente.

9.2.4 Opérations de pile

`av-vector-push` et `av-vector-pop` sont maintenant corrects avec `fill-pointer` = 0, y compris dans le benchmark dédié.

Exemples du run de référence :

- taille 1000 : `av-vector-push-pop` 143 ms, `cons-cdr` 0 ms ;
- taille 5000 : `av-vector-push-pop` 142 ms, `cons-cdr` 0 ms.

Conclusion : la sémantique est stable, mais le coût reste très supérieur à la pile native sur listes, ce qui est attendu pour cette v1.

9.2.5 Sous-séquences

`av-subseq` reste plus lent que `list-subseq`, mais le changement de représentation a encore réduit le coût des accès répétés.

Exemples du run de référence :

- taille 500 : `av-subseq` 749 ms, `list-subseq` 3 ms ;
- taille 1000 : `av-subseq` 789 ms, `list-subseq` 3 ms ;
- taille 5000 : `av-subseq` 1040 ms, `list-subseq` 3 ms.

Conclusion : l'amélioration est réelle, mais la constante d'accès par case reste trop élevée pour rendre l'opération compétitive.

9.2.6 Remplacement de tranche

`av-replace` a lui aussi bénéficié de l'itération séquentielle, mais reste de loin l'opération la plus coûteuse sur la représentation courante.

Exemples du run de référence :

- taille 500 : `av-replace-mid` 2186 ms, `list-replace-mid` 4 ms ;
- taille 1000 : `av-replace-mid` 2293 ms, `list-replace-mid` 4 ms ;
- taille 5000 : `av-replace-mid` 2991 ms, `list-replace-mid` 4 ms.

Conclusion : la copie et les accès répétés au chemin critique doivent être revus avant d'espérer une compétitivité pratique.

9.3 Lecture d'ensemble

La baseline du 14 mars 2026 établit les points suivants :

- la sémantique publique de `v1` est correcte ;
- le protocole de benchmark est exploitable et reproductible ;
- `av-length` atteint bien son objectif ;
- `av-set-aref` devient meilleur que `list-set-at` dès les tailles moyennes ;
- `av-subseq` et `av-replace` ont beaucoup progressé mais restent trop coûteux ;
- `av-aref` reste derrière `nth` mais dans un ordre de grandeur bien plus raisonnable.

9.4 Interprétation technique

L'explication la plus probable du coût résiduel est la combinaison de :

- profondeur logarithmique de l'arbre interne ;
- reconstruction persistante du chemin sur chaque écriture ;
- parcours répétés case par case pour `av-subseq` et `av-replace` ;
- comparaison avec les primitives de liste natives de l'interpréteur.

Cette baseline doit donc être considérée comme :

- une **baseline fonctionnelle valide** ;
- une **baseline asymptotique convaincante** pour `length` et `set-aref` ;
- mais **pas encore** une baseline de performance satisfaisante pour `aref`, `subseq` et `replace`.

9.5 Décision de gel

À partir de cette date, cette v1 sert de point de comparaison pour toute évolution ultérieure, notamment :

- simplification de la représentation interne ;
- réduction du nombre d'indirections dans `av-aref` et `av-set-aref` ;
- éventuelle étude d'un facteur de branchement > 2 ;
- comparaison avant/après sur le même protocole `bench-bricscad`.

10 Plan de réalisation

10.1 Étape 1

Définir les primitives et invariants du vecteur fixe :

- création ;
- lecture ;
- écriture ;
- `fill-pointer` ;
- `replace` ;
- `subseq` ;
- `print-vector` ;
- validation des indices.

10.2 Étape 2

Ajouter les tests unitaires et les tests d'invariants.

10.3 Étape 3

Ajouter les micro-benchmarks et comparer plusieurs facteurs de branchement.

10.4 Étape 4

Décider si la future table de hashage réutilise directement cette structure ou une variante plus large adaptée aux chemins de hash.